

Timer-Ersatzplatine — Vollständige Dokumentation

Projekt: Ersatz einer defekten/veralteten Timer-Steuerplatine durch eine Eigenentwicklung mit ESP32-C3, OLED-Display, WLAN-Konfiguration und Shelly 1PM Mini Gen4 als Leistungsschalter.

Stand: 2026-07-13 · Vorlagenversion v8 · Phase: Verifikation vor Layout **Autor:** Silvio (Hardware/Messung) + Claude (Auswertung/Konstruktion)

1. Was das Gerät tut (Funktionsbeschreibung)

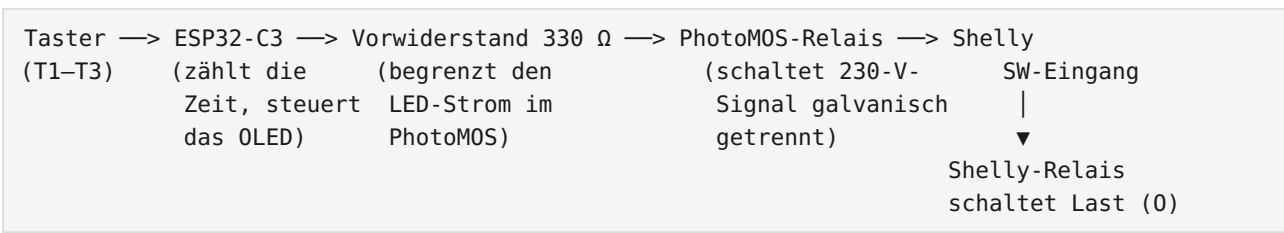
Das Gerät hat drei Taster an der Frontseite. Ein Druck auf einen Taster schaltet eine 230-V-Last für eine einstellbare Zeit ein:

Taster	Standardzeit	einstellbar
T1	5 Sekunden	1-600 s über WLAN
T2	10 Sekunden	1-600 s über WLAN
T3	15 Sekunden	1-600 s über WLAN

Ein kleines OLED-Display hinter der Sichtscheibe zeigt oben den **WLAN-Empfang** (links), die große **Uhrzeit** (NTP, Mitte) und den Status **Ruhe** (rechts); unten links das **Datum** (z. B. „Do 16.07.2026“) und rechts den **freien Speicher**. Läuft ein Timer, steht in der Mitte der **Sekunden-Countdown** und rechts **Futter**. Die drei Tasten (**S1 Down/Manual**, **S2 SET**, **S3 UP**) lösen Timer aus, stoppen und blättern durch ein Info-Menü — Details in Kap. 6.5.

Die Bedienung läuft **ohne App und ohne Cloud** über eine kleine, für Mobilgeräte optimierte Webseite, die der ESP32 selbst auf **Port 80** bereitstellt (im Browser `http://feeder-relais.local` aufrufen): Taster auslösen, Zeiten einstellen, Netzwerk- und Statuswerte ansehen. Für die Anbindung (z. B. ioBroker) bietet der ESP zusätzlich eine schlanke **JSON-API** (siehe Kap. 6.4). Optional lässt sich das Gerät auch in Home Assistant oder ioBroker (esphome-Adapter) einbinden.

1.1 Die Signalkette (so fließt das Signal)



Warum ein Shelly? Der Shelly 1PM Mini Gen4 übernimmt das eigentliche Schalten der Last, misst dabei die Leistung und ist zusätzlich per WLAN/App erreichbar. Unsere Platine „drückt“ für den Shelly nur den Schalter — als echtes Hardware-Signal, **ohne WLAN-Abhängigkeit** für die Kernfunktion.

Was ist ein PhotoMOS? Ein winziges Halbleiterrelais: Auf der einen Seite eine LED (die der ESP mit 3,3 V ansteuert), auf der anderen Seite ein lichtgesteuerter Schalter, der Netzspannung schalten darf. Dazwischen liegt nur Licht — also eine vollständige galvanische Trennung zwischen Kleinspannung und 230 V. Der SW-Eingang des Shelly ist netzspannungs- bezogen (erwartet

geschaltetes L), deshalb **zwingend ein Typ mit $\geq 400\text{ V}$ Sperrspannung** (z. B. Omron G3VM-601BY oder Panasonic AQY216). Ein gewöhnlicher Optokoppler (PC817) ist hier **nicht** zulässig.

Der Shelly wird auf Eingangsmodus „**Schalter/Follow**“ konfiguriert: sein Relais ist genau so lange an, wie der ESP das Signal hält. Das Timing liegt vollständig beim ESP.

2. Sicherheit — bitte zuerst lesen

- Das Gerät arbeitet mit **230 V Netzspannung. Lebensgefahr!** Aufbau, Prüfung und Inbetriebnahme gehören in die Hände einer Person mit entsprechender Qualifikation (Elektrofachkraft oder unterwiesen).
 - Erste Funktionstests von ESP, Tastern und OLED erfolgen **ausschließlich über USB** — dabei ist keinerlei Netzspannung verbunden (siehe Testplan im Projektplan).
 - Der erste 230-V-Test erfolgt hinter einem **RCD/PRCD (Personenschutz- stecker)**, Gehäuse geschlossen, niemals unter Spannung im offenen Gerät hantieren.
 - Auf der Platine gilt: zwischen allen 230-V-Netzen und der Kleinspannung **mindestens 6 mm Abstand** (Kriechstrecke), Details in Kap. 7.6.
 - Sicherung (1 A träge) und Varistor vor dem Netzteilmodul sind Pflicht — die Module (TSP-05/HLK) bringen beides nicht selbst mit.
-

3. Stückliste (BOM)

Ref	Bauteil	Wert / Typ	Bemerkung
U1	ESP32-C3 Super Mini	18 × 24 mm, USB-C	Controller, WLAN
U2	PhotoMOS-Relais	Omron G3VM-601BY oder Panasonic AQY216	$\geq 400\text{ V!}$ SOP-4/DIP-4
PS1	AC/DC-Modul	TENSTAR TSP-05 (5 V / 3 W)	HLK-PM05-Klon, 34,7 × 20,5 × 15,05 mm, Pinabstände nachmessen!
—	Shelly 1PM Mini Gen4	S4SW-001P8EU	29 × 34 × 16 mm, max. 8 A/240 V
—	OLED-Display	0,91" SSD1306, 128 × 32, I2C	Modul 38 × 12 mm, Pins: GND VCC SCL SDA, Adresse 0x3C
SW1-3	Kurzhubtaster	6 × 6 mm, Hub 1,5 mm	Pinbild 6,9 × 4,4 mm (aus Original vermessen)
R1	Widerstand	330 Ω , axial	LED-Vorwiderstand PhotoMOS
F1	Feinsicherung	1 A träge, 5 × 20 mm + Halter	primärseitig
RV1	Varistor	S14K275	primärseitig parallel
C1	Elko	220 μF / 10 V	5-V-Puffer für WLAN-Stromspitzen
C2	Kerko	100 nF	5-V-Abblockung
J1	Netzklemme	2-polig (L, N)	Schraubklemme oder Löt pads
J2	Shelly-Anschluss	4 Löt pads/Stifte	L, N, SW, O — kurze Litzen zu den Shelly- Schraubklemmen
J3	Lastabgang	2-polig	O (geschaltet), N
J4	OLED-Buchse	Buchsenleiste 1 × 4, RM 2,54	Bauhöhe nach Stackmessung ($\sim 11\text{ mm}$), Kap. 5.6

Ref	Bauteil	Wert / Typ	Bemerkung
—	Stiftleiste 1 × 4	RM 2,54	wird ans OLED gelötet (nach hinten)
—	Montagematerial	3M-VHB-Klebeband o. 3D-Clip	Befestigung Shelly + TSP

4. Gehäuse und Mechanik

4.1 Das Koordinatensystem (wichtig für alles Weitere!)

Alle Maße beziehen sich auf die **obere linke Ecke der ORIGINALPLATINE**, x nach rechts, **y nach unten**, Blick auf die **Bestückungsseite** (Taster oben). Die neue, größere Platine ragt über diesen Ursprung hinaus — deshalb gibt es **negative Koordinaten**. Vorteil: Alle je gemessenen Werte bleiben für immer gültig, egal wie die Kontur wächst.

4.2 Randbedingungen aus dem Gehäuse (gemessen/abgeleitet)

Größe	Wert	Quelle
Innere Gesamthöhe	19,9 mm	Messung Silvio
Rahmenhöhe (Frontplatte)	2 mm	Messung Silvio
Bauraum über der Platine	16,5 mm	Messung Silvio (Platine ↔ Front)
Platinenstärke	1,6 mm	Standard
Raum unter der Platine	≈ 1,8 mm	19,9 – 16,5 – 1,6 → nichts darf nach unten ragen!
Höhenlimit im Fensterbereich	≈ 12 mm	Glas + davor gestecktes OLED
Platinenbefestigung	4 Schrauben auf 16,5-mm-Domen an der Frontplatte	mit Stabilisierungsstegen zum Rand
Gehäuseverschraubung	6 Schrauben außenliegend	Platine hat 6 randoffene U-Schlitze (Breite 10) statt geschlossener Durchführungen

4.3 Finale Platinengeometrie (Vorlage v9)

Alle Werte stehen **maschinenlesbar** in `hardware/platine_template.py` (die einzige Quelle der Wahrheit!). Zusammenfassung:

Element	Koordinaten (mm)	Genauigkeit
Kontur	x –14,2...91,1 / y –11,8...67,6 → 105,3 × 79,4 , Eckenradius 4 (an den äußeren Dom-Schlitzen lokal auf 3,2 bzw. 2,1 reduziert)	±1 (links ±1,5)
4 Befestigungslöcher ø3,0	(3,8/4,1) (73,1/4,0) (3,7/39,6) (73,4/50,7)	±0,3
6 Dom-Schlitze (randoffene U, Breite 10)	obere Reihe y –6,0 (öffnet zur Oberkante), untere Reihe y +64,0 (öffnet zur Unterkante), Reihenabstand 70; Spalten x –6,0 / 39,0 / 84,0 (Pitch 45)	direkt vermessen (Silvio 13.07.); Absolutlage noch anprobieren
Taster T1/T2/T3 (Zentren)	(13,2/12,3) (38,8/12,2) (64,1/12,0)	±0,3

Element	Koordinaten (mm)	Genauigkeit
Sichtfenster (Referenz)	x 18,4...53,6 / y 23,3...40,0	±1
J4 OLED-Buchse (4 Pins senkr.)	Pins bei x = 15,0, y = 27,84 / 30,38 / 32,92 / 35,46	Annahmen prüfen, Kap. 5.6
Stellfläche Shelly	x 56...90 / y 16,5...45,5 (liegend)	—
Stellfläche ESP32-C3	x 8,5...32,5 / y -10...8 (quer, USB nach oben)	—
Stellfläche TSP-05	x -1,5...33,2 / y 44,2...64,7 (AC-Seite nach links)	—
Keepouts (Referenz)	ø8 um die 4 Schraubenköpfe + 5 Steg-Bänder	Stegrichtungen = Annahme!

Woher stammen die Werte? Lötseiten-Scan der Originalplatine (300 dpi, entdreht, gespiegelt) für Taster/Löcher/Kontur; Gehäusescan, registriert über die 4 Rahmenschrauben (Spiegel-Transformation $x_{\text{platine}} = 110,5 - x_{\text{gehäuse}}$, $y_{\text{platine}} = y_{\text{gehäuse}} - 16,4$, $\pm 0,4$ mm); Fensterposition aus dem perspektivisch entzerrten Foto.

4.4 Die alte Aussparung

Die trapezförmige Aussparung an der Unterkante der Originalplatine war die Freistellung des mittleren unteren Schraubdoms (B5). Sie ist in der neuen Platine **durch den randoffenen U-Schlitz B5 ersetzt** und entfällt.

5. Elektrik im Detail

5.1 Übersicht der Netze (Verbindungen)

Netzname	führt	verbindet
L_IN	230 V	Netzklemme J1.L → Sicherung F1
L_F	230 V (abgesichert)	F1 → Varistor, TSP-05 AC, PhotoMOS OUT1, Shelly L
N	230 V Neutral	J1.N → Varistor, TSP-05 AC, Shelly N, Last J3
SW_SHELLY	230 V geschaltet	PhotoMOS OUT2 → Shelly SW
O_LAST	230 V geschaltet	Shelly O → Lastabgang J3
+5V	Kleinspannung	TSP-05 +Vo → ESP 5V, C1, C2
GND	Kleinspannung	TSP-05 -Vo → ESP GND, Taster, PhotoMOS K, OLED
+3V3	Kleinspannung	ESP-3V3-Ausgang → OLED VCC
BTN1/2/3	Signal	Taster → GPIO3/4/5 (interne Pullups)
PMOS_DRV → PMOS_A	Signal	GPIO6 → R1 330 Ω → PhotoMOS-Anode
SDA / SCL	I2C	GPIO7 / GPIO9 → OLED
RGB-LED	Signal	GPIO8 → Onboard-WS2812 (Status-LED, gedimmt)

5.2 ESP32-C3-Super-Mini: Board und Pinbelegung

Modul-Variante „ESP32-C3FN4“. Die von uns genutzten Pins im Überblick:

ESP32-C3 Super Mini — GPIO-Belegung (Feeder-Relais)



GPIO3 Taster S1 — Down / Manual

GPIO4 Taster S2 — SET

GPIO5 Taster S3 — UP

GPIO6 PhotoMOS-Treiber (330 Ω) -> Shelly SW

GPIO7 I2C SDA -> OLED (war GPIO8!)

GPIO8 RGB-Status-LED (onboard WS2812)

GPIO9 I2C SCL -> OLED

5V/G/3V3 Versorgung (TSP-05) + OLED-VCC

Bauteile auf dem Modul: **1** USB-Type-C · **2** BOOT-Taster · **3** RESET-Taster · **4** LDO CAT6219 (3,3 V, 500 mA) · **5** ESP32-C3FN4 · **6** 2,4-GHz-Antenne · **7 RGB-Modul (WS2812, GPIO8)** · **8** Antennen-Anschluss (u.FL).

Pin	Funktion	Anmerkung
5V / G	Versorgung	vom TSP-05
3V3	Ausgang des Onboard-LDO	versorgt das OLED
GPIO3	Taster T1	Pullup intern, Taster nach GND
GPIO4	Taster T2	dito
GPIO5	Taster T3	dito
GPIO6	PhotoMOS-Treiber	high = Shelly ein
GPIO7	SDA (I2C)	OLED — statt GPIO8 , dort sitzt die RGB-LED
GPIO8	RGB-LED (WS2812)	Onboard-Status-LED, gedimmt
GPIO9	SCL (I2C)	OLED

Bewusst vermieden: GPIO2/8/9 als Taster (Strapping-Pins). **GPIO8 trägt die Onboard-RGB-LED (WS2812)** — deshalb liegt I2C-SDA auf GPIO7 (nicht GPIO8), sonst würde der I2C-Verkehr die LED hell ansteuern (Hitze). SCL bleibt GPIO9 (mit I2C-Pullup bootkompatibel).

5.3 Shelly 1PM Mini Gen4

- Klemmen: **SW** (Schalteingang), **O** (Lastausgang), **L**, **N**.
- Versorgung 110-240 V~, Relais max. 8 A / 2000 W @ 240 V.
- Konfiguration nach Inbetriebnahme: Input Mode = Switch (Follow), damit das Relais dem SW-Signal 1:1 folgt.
- Montage: liegend auf der Stellfläche, VHB-Klebeband oder 3D-Clip. Vier kurze Litzen (z. B. 0,75 mm², 6-7 mm abisoliert) von den Schraubklemmen zu den J2-Pads.

5.4 Netzteil TSP-05

100–240 V~ → 5 V DC / 3 W (= 600 mA). Reicht für ESP-WLAN-Spitzen (~350 mA) plus OLED mit Reserve. **Offen: Pinabstände mit Messschieber gegen den KiCad-Footprint**

Converter_ACDC_HiLink_HLK-PMxx prüfen (Klon ist üblicherweise pinkompatibel — bei 230 V nichts annehmen).

5.5 Taster

Die drei 6×6-Kurzhubtaster (1,5 mm Hub) werden von Stößeln in der Frontplatte betätigt — ihre Position ist deshalb **nicht verhandelbar** (auf ±0,2 mm bestücken). Pinbild je Taster: 6,9 mm × 4,4 mm.

5.6 OLED — Steckmontage („Stack“)

Das OLED wird **nicht verlötet**, sondern **auf J4 aufgesteckt**: Buchsenleiste 1×4 auf der Platine, Stiftleiste am OLED nach hinten, Display-Face liegt (mit dünnem Schaumstoffpad) an der Glasscheibe an.

- Belegung J4 von oben nach unten: **GND, VCC (3V3), SCL, SDA** — identisch mit der Beschriftung des vorhandenen Moduls.
- **Benötigte Stackhöhe** = 16,5 mm – Glaseinstand – Modulstärke (≈ 2,8 mm) ≈ **13–14 mm**. Realisierung z. B. Buchsenleiste 11 mm + Stiftleisten- kragen 2,5 mm. Buchsenleisten gibt es in 8,5/11/13/15 mm.
- **Vor dem Einfrieren am realen Modul messen** (Hersteller streuen ±1 mm): 1. Abstand Modul-Linkskante → Pinreihe (Annahme 2,5 mm), 2. Abstand Modul-Linkskante → Zentrum aktive Fläche (Annahme 23,5 mm), 3. Glaseinstand (Front-Innenfläche → Glasunterseite). Praktisch: Modul in den Rahmen legen, Display mittig im Fenster, Pinlöcher relativ zu zwei Dom-Schrauben messen → das ist J4.

6. Firmware (ESPHome)

Datei: `firmware/timer-relais-c3.yaml` (Gerätename **feeder-relais**, mDNS `feeder-relais.local`). Enthält komplett: WLAN mit Fallback-Hotspot („Feeder-Relais Setup“, Captive-Portal), **NTP-Uhr** (`de.pool.ntp.org`), mobile Web-App + JSON-API (Kap. 6.4), die drei persistenten Zeiten, Tasterlogik mit Entprellung/Langdruck/Menü (Kap. 6.5) und ein OLED mit WLAN-Status, Uhr bzw. Countdown.

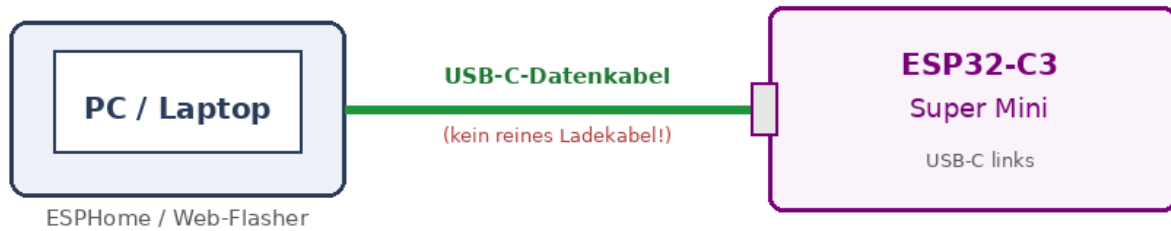
6.1 Erstes Flashen — Schritt für Schritt (für Einsteiger)

Beim **allerersten Mal** kommt die Firmware **per USB-Kabel** auf den ESP32-C3. Alle späteren Updates laufen dann drahtlos (Kap. 6.6).

Das brauchst du:

- den **ESP32-C3 Super Mini**,
- ein **USB-C-Datenkabel** — Achtung: viele billige Kabel können nur laden; damit wird der Chip **nicht erkannt!**
- einen PC mit **Google Chrome** oder **Microsoft Edge** (Browser-Weg B) — oder ESPHome auf dem PC (Weg A).

Schritt 1 - Verbinden



Fertige Images liegen bereit — du musst also nicht selbst kompilieren:

- im Repo unter `firmware/build/` (immer der aktuelle Stand):
- `feeder-relais.factory.bin` — Voll-Image für den **Erstflash** (Adresse 0x0)
- `feeder-relais.ota.bin` — App-Image für **OTA/Web-Update** (Kap. 6.6)
- als **GitHub-Release** (versionierter Dateiname, z. B. `feeder-relais-v0.0.1.factory.bin`) im Bereich Releases des Repositories.

Selbst neu bauen: `./firmware/build_images.sh` (kompiliert und aktualisiert `firmware/build/`). Die Roh-Dateien liegen sonst unter `firmware/.esphome/build/feeder-relais/.pioenvs/feeder-relais/`.

Die Images enthalten KEINE WLAN-Zugangsdaten. Die Einrichtung läuft nach dem Flashen über den Setup-Hotspot (Kap. 6.2). Damit kommt kein fremdes WLAN in die Auslieferung, und die eingerichteten Daten überstehen OTA-Updates (fester Speicher-Schlüssel).

Weg A — mit ESPHome (empfohlen, ein Befehl)

Eine `secrets.yaml` ist **nicht** nötig — es werden keine WLAN-Daten einkompiliert.

1. ESP per **USB-C** anstecken.
2. Im Projektordner ausführen: `esphome run firmware/timer-relais-c3.yaml` ESPHome kompiliert, fragt den **seriellen Port** ab (Linux z. B. `/dev/ttyACM0`, Windows ein `COMx`) und flasht.
3. Nach dem Neustart öffnet der ESP den Setup-Hotspot → weiter mit Kap. 6.2.

Linux-Tipp: Bei „Permission denied“ auf `/dev/ttyACM0` deinen Benutzer in die Gruppe `dialout` aufnehmen: `sudo usermod -aG dialout $USER` (neu anmelden).

Weg B — im Browser, ohne Installation (esptool-js)

Für die fertige `firmware.factory.bin` ganz ohne ESPHome:

Ablauf - Erstes Flashen ueber den Browser (esptool-js)

- 1 Chrome oder Edge oeffnen**
espressif.github.io/esptool-js
- 2 ESP per USB-C anstecken**
USB-C-Datenkabel verwenden
- 3 "Connect" -> Port waehlen**
ESP32-C3 als serieller Port
- 4 Flash Address 0 + factory.bin**
Datei waehlen, dann "Program"
- 5 WLAN einrichten**
AP "Feeder-Relais Setup" -> feeder-relais.local

1. In **Chrome/Edge** die Seite <https://espressif.github.io/esptool-js/> öffnen.
2. ESP per **USB-C-Datenkabel** anstecken.
3. Baudrate **115200** lassen, **Connect** klicken und im Fenster den seriellen Port des ESP wählen (heißt oft „USB JTAG/serial debug unit“ oder ein COMx).
4. Bei **Flash Address** **0** eintragen, mit **Choose File** die `firmware.factory.bin` wählen, dann **Program**. Warten, bis „Hard resetting...“ erscheint.
5. Weiter mit der WLAN-Einrichtung (Kap. 6.2).

Klappt „Connect“ nicht: **Boot-Modus** erzwingen — **BOOT** gedrückt halten, kurz **RESET** tippen, **BOOT** loslassen, dann erneut „Connect“.

6.2 Nach dem Flashen: WLAN einrichten

Die Firmware bringt **absichtlich keine** WLAN-Zugangsdaten mit — nach dem Flashen startet der ESP einen eigenen **Setup-Hotspot**:

1. Am Handy/PC ins WLAN „**Feeder-Relais Setup**“ gehen (Passwort `feeder1234`).
2. Es öffnet sich ein **Captive-Portal** (sonst <http://192.168.4.1> aufrufen).
3. Dein Heim-WLAN auswählen, Passwort eingeben, speichern — der ESP startet neu und verbindet sich.
4. Ab jetzt erreichbar unter <http://feeder-relais.local>.

Persistenz & Werksreset:

- Die eingegebenen WLAN-Daten liegen unter einem **festen Speicher-Schlüssel** und bleiben bei **OTA-/Web-Updates erhalten** (Kap. 6.6) — einmal einrichten, dann nie wieder.
- Für einen **Werksreset** (alte WLAN-Daten löschen) das **Factory-Image mit Flash-Erase** flashen: im Web-Flasher „Erase all flash“ ankreuzen bzw. `esptool.py erase_flash` vor dem Schreiben. Ein reines OTA-Update löscht die Daten **nicht**.

6.3 Problemlösung beim Flashen

Symptom	Ursache / Lösung
ESP wird gar nicht erkannt	Ladekabel statt Datenkabel → anderes USB-C-Kabel, anderer USB-Port
Kein Port im Browser	Chrome/Edge nutzen (WebSerial); Windows ggf. CH340-/CP210x -Treiber; C3 mit nativem USB braucht meist keinen
„Connect“ schlägt fehl	Boot-Modus : BOOT halten + RESET tippen + BOOT loslassen
Linux „Permission denied“	Benutzer in Gruppe dialout (sudo usermod -aG dialout \$USER)
Nach dem Flash keinlocal	erst WLAN über den Setup-Hotspot einrichten (Kap. 6.2); mDNS braucht einen Moment

Nach dem Erstflash gehen alle Updates **drahtlos** (Kap. 6.6) — die USB-Buchse darf im eingebauten Zustand ruhig schlecht erreichbar sein.

6.4 Web-Bedienung und JSON-API (Port 80)

`http://feeder-relais.local` öffnet eine mobile Web-App mit fünf Reitern:

- **Start**: drei große Taster (lösen T1/T2/T3 mit ihrer eingestellten Zeit aus), Live-Countdown und **Stopp**.
- **Zeiten (Einstellungen)**: ganz oben die **Sprachauswahl** (Deutsch/English/Français), darunter die drei Timerzeiten (1–600 s), persistent gespeichert.
- **Netzwerk (konfigurierbar)**: WLAN (SSID/Passwort), IP-Modus **DHCP/statisch** (+ IP/Gateway/Maske/DNS), **NTP-Server**, ein **Hostname** (Default `feeder-relais`), ein Schalter **WLAN-Roaming (802.11k/v)** und ein **Neustart**-Knopf.
- **Status**: ganz oben die geflashte **Version**, dann Firmware-Baustand, Laufzeit, freier Speicher, WLAN, SSID mit **Kanal · farbigem Signalbalken · Signal** (dBm; 1 Balken rot, 2 gelb, 3–4 grün), **WLAN-Roaming (802.11k/v) EIN/AUS**, IP/MAC, Reset-Grund, Relais.
- **Service**: Live-**Log** (Anzeige-Stufe ERROR/WARN/INFO/DEBUG wählbar, aktivierbar), **Firmware-Update** (.bin-Upload, Kap. 6.6) und **Neustart**.

Sprache (mehrsprachige Oberfläche): Die Web-App und das OLED sprechen **Deutsch, Englisch und Französisch**. Die Auswahl steht **ganz oben im Reiter Zeiten** und gilt **geräteweit** (Web und OLED), gespeichert in `g_netcfg.lang` (persistent, Default Deutsch). Die Web-Texte liegen als JS-Wörterbuch in der Seite (Umschalten ohne Neuladen); sprachabhängige Statuswerte (WLAN-Zustand, Reset-Grund) kommen als neutrale Codes aus der Firmware und werden im Browser übersetzt. Das OLED übersetzt Wochentage, den Status (Ruhe/Futter) und die Menütitel (bewusst akzentfrei wegen des Display-Zeichensatzes). Das Handbuch selbst gibt es zusätzlich in [Englisch](#) und [Französisch](#) (die deutsche Fassung ist maßgeblich).

In der **Kopfzeile** stehen der **Hostname** und ein **Statuspunkt**: grün = alles in Ordnung, gelb = ein Timer läuft, rot = Störung im Ruhezustand (z. B. OLED nicht erreichbar oder kein WLAN). **Dieselbe Ampel** zeigt am Gerät die gedimmte **Onboard-RGB-LED** (WS2812 auf GPIO8, siehe Kap. 5.2).

Technisch liefert `firmware/timer_web.h` (mit `firmware/net_config.h` für die persistente Netzwerk-Konfig) diese Seite als eigener `AsyncWebHandler` auf `web_server_base` (das native ESPHome-Web-UI ist deaktiviert). Dieselbe **JSON-API** nutzt auch die Anbindung (z. B. `ioBroker`) — Parameter als Query-String, Methode GET **oder** POST:

Endpoint	Wirkung
GET /api/status	JSON mit allen Werten: active, remaining, relay, last, times[3], host, ip, ssid, rssi, mac, ap, fw, ver, uptime, heap, wifi, reset, lang, roaming (ver = geflashte Version; wifi/reset sind sprachneutrale Codes, die das Web-JS übersetzt)
POST /api/trigger?button=N	Taster N (1-3) auslösen (nutzt dessen eingestellte Zeit)
POST /api/trigger?seconds=N	ad-hoc für N Sekunden schalten
POST /api/stop	sofort abschalten
POST /api/config?time1=A&time2=B&time3=C	Zeiten setzen (je 1-600 s, persistent) — Felder auch einzeln
GET /api/net	Netzwerk-Konfig lesen: static, ip, gw, sn, dns, ntp, hostname, roaming, lang
POST /api/net?static=0\ 1&ip=&gw=&sn=&dns=&ntp=&host=&roaming=0\ 1&lang=de\ en\ fr	Netzwerk-Konfig speichern (Felder einzeln, persistent)
POST /api/wifi?ssid=&pw=	WLAN-Zugangsdaten setzen (verbindet neu)
POST /api/reconnect	WLAN sauber neu verbinden (übernimmt u. a. geändertes Roaming sofort)
POST /api/reboot	Gerät neu starten
GET /api/log?level=N&since=M	Log-Ringpuffer als JSON (Zeilen mit Level ≤ N, seq > M); Level 1=ERROR...5=DEBUG

Anwendung der Netzwerk-Konfig: Die Werte liegen in den ESPHome-Preferences (Flash). **NTP-Server** wirkt beim nächsten Sync (`esp_ntp_setservername`). **Statische IP** wird beim `wifi.on_connect` per ESP-IDF-netif gesetzt und ist deshalb nach einem **Neustart** aktiv (DHCP ist Default). **WLAN** nutzt ESPHomes eigenes `save_wifi_sta()`; Erstzugang immer übers Captive-Portal. **Hostname** wird zur Laufzeit umgestellt: mDNS (...local) sofort per `mdns_hostname_set()`; der DHCP-Name (im Router) durch einen **Neustart des DHCP-Clients** (`esp_netif_dhpcpc_stop/start` → frisches DISCOVER mit Option 12), also ebenfalls ohne Reboot. Der Name erscheint auch in der **Kopfzeile** der Web-App und im Browser-Tab. Er wird auf ein gültiges DNS-Label reduziert (a-z, 0-9, „-“).

WLAN-Roaming (802.11k/v): Die Fähigkeit ist fest einkompiliert (`enable_btm/enable_rrm` in der YAML → `CONFIG_WPA_11KV_SUPPORT` im `wpa_supplicant`); das **Ein/Aus** schaltet der Web-Schalter zur Laufzeit über `g_netcfg.roaming` (persistent). Ein setzt in der STA-Config die Bits 802.11v **BTM** (`set_btm`, der AP/Router kann das Gerät gezielt auf den stärkeren AP umbuchen) und 802.11k **RRM** (`set_rrm`, Nachbar-AP-Listen) und schaltet ESPHomes eigenes Scan-Roaming ab (Treiber übernimmt); Aus macht es umgekehrt (`set_post_connect_roaming(true)`). Sinnvoll nur bei **mehreren Access-Points mit gleicher SSID** (UniFi/Mesh) und wenn diese 802.11k/v unterstützen. Die Bits gehen erst beim **nächsten (Re)Connect** in die STA-Config; dafür gibt es in derselben Karte den Knopf „**Jetzt neu verbinden**“ (`POST /api/reconnect` → `wifi.disable+enable`, das WLAN trennt kurz), sonst wirkt es beim nächsten Neustart. Default: **aus**. Der Haken spiegelt **laufend den echten Gerätezustand** (`aus /api/status`) und bleibt daher nach dem Neuverbinden sichtbar gesetzt; der aktuelle Zustand steht zusätzlich als Zeile „**WLAN-Roaming (802.11k/v): EIN/AUS**“ auf der **Status-Seite**.

Persistenz (wichtig): Die Web-Handler laufen im **Webserver-Task**. ESPHomes Preferences (globale Warteschlange + NVS) sind **nicht threadsicher** gegen den Main-Task — ein Flash-

Schreiben (`save()`/`sync()`) direkt aus dem Handler verursachte **Panik-Neustarts**. Deshalb setzt `netcfg_save()` nur ein Flag; das eigentliche, dauerhafte Schreiben (`netcfg_flush()` → `global_preferences->sync()`) erledigt das **1-s-Interval im Main-Task** (dieselbe Task wie ESPHomes eigener Preferences-Sync). Das gilt für **alle** Netzwerk-Einstellungen.

Beispiel ioBroker (Zeit Taster 1 auf 8 s setzen): `POST http://feeder-relais.local/api/config?time1=8`.

6.5 Bedienung an den drei Tasten und OLED

Die Tasten sind am Gehäuse beschriftet: **S1 = Down/Manual**, **S2 = SET**, **S3 = UP** (GPIO3/4/5).

Normalzustand:

- **kurz S1 / S2 / S3** → löst Timer 1 / 2 / 3 mit der eingestellten Zeit aus (schaltet den Shelly an, bis die Zeit abläuft).
- **lang UP (S3) ≥ 1,2 s** → stoppt alle Timer und schaltet den Shelly **aus**.
- **lang SET (S2) ≥ 3 s** → öffnet das **Info-Menü**.

Info-Menü (nur Ansicht — die Konfiguration läuft über die Web-App):

- **S1** blättert vor, **S3** zurück; **kurz SET** oder **10 s ohne Druck** schließt das Menü wieder.
- Seiten: 1) WLAN (SSID + Signal) · 2) IP-Adresse · 3) Zeit + NTP · 4) System (Firmware + Laufzeit).

OLED-Anzeige (128 × 32):

- **Oben:** WLAN-Signalbalken (aus RSSI) links, die große **Uhrzeit** (HH:MM) bzw. bei laufendem Timer der **Countdown** (z. B. „10 s“) in der Mitte, Status **Ruhe/Futter** rechts; im Menü stattdessen der Seitenzähler (z. B. „2/4“).
- **Unten:** links das **Datum** (z. B. „Do 16.07.2026“), rechts der **freie Speicher** (kB).

Die Zeitbasis kommt per NTP; bis zur ersten Synchronisation zeigt die Uhr „--:--“. Zeitzone `Europe/Berlin`. Wochentag, Status (Ruhe/Futter) und die Menütitel folgen der **eingestellten Sprache** (Kap. 6.4).

6.6 Firmware-Updates (OTA)

Nach dem ersten USB-Flash sind **zwei drahtlose Update-Wege** eingerichtet:

- **Netzwerk-OTA (ESPHome):** `esphome run firmware/timer-relais-c3.yaml` aktualisiert über WLAN (Port 3232) — USB nicht mehr nötig. Konfiguriert über `ota: platform: esphome`.
- **Web-Upload:** Im **Service**-Tab die kompilierte `firmware.bin` hochladen (bzw. direkt `POST /update` auf Port 80). Konfiguriert über `ota: platform: web_server` — läuft über `web_server_base`, das native ESPHome-Web-UI wird dafür **nicht** gebraucht. Das Gerät startet nach dem Update automatisch neu.

Die `firmware.bin` entsteht bei `esphome compile ...` und liegt unter `.esphome/build/feeder-relais/.pioenvs/feeder-relais/firmware.bin`. Auch das Captive-Portal kann Updates einspielen.

6.7 Service-Log / Debugging

Der **Service**-Tab zeigt bei aktivierter „Live-Anzeige“ die letzten Log-Zeilen (Ringpuffer, 40 Zeilen) gefiltert nach gewählter Stufe (ERROR/WARN/INFO/DEBUG). Technik: `logger: on_message:` schreibt jede Zeile in `firmware/log_ring.h`; der Web-Endpunkt `GET /api/log?level=&since=` liefert sie als

JSON. Der Logger läuft auf `level: DEBUG` (für VERBOSE die Stufe in der YAML anheben). Vollständige Logs zusätzlich per `esphome logs ...` (UART/Netz).

7. KiCad — Schritt für Schritt (auch für Einsteiger)

Getestet gegen KiCad 8/9/10; Menünamen können minimal abweichen.

7.1 Projekt anlegen und Schaltplan übernehmen

1. KiCad starten → **Datei** → **Neues Projekt** → Name `timer_ersatzplatine`, Ordner z. B. `hardware/kicad/`.
2. KiCad erzeugt eine leere `timer_ersatzplatine.kicad_sch`. Diese Datei **im Explorer durch unsere Datei ersetzen** (gleicher Name!): `hardware/timer_ersatzplatine.kicad_sch` hineinkopieren.
3. Projekt öffnen, Schaltplan-Editor starten. KiCad konvertiert die Datei ggf. ins aktuelle Format — das ist normal, einfach speichern.
4. **Werkzeuge** → **Elektrische Regeln prüfen (ERC)** laufen lassen. Warnungen über „nicht angeschlossene Power-Pins“ sind bekannt und unkritisch (unsere Symbole haben bewusst keine Power-Pin-Typen); optional mit `PWR_FLAG`-Symbolen auf +5V und GND stilllegen.

7.2 Footprints prüfen/zuordnen

Werkzeuge → **Footprints zuweisen**. Vorbelegt sind:

Ref	Footprint	Aktion
SW1-3	Button_Switch_THT:SW_PUSH_6mm	prüfen (Pinbild 6,9 × 4,4)
PS1	Converter_ACDC:Converter_ACDC_HiLink_HLK-PMxx	gegen TSP-05-Messung prüfen!
U2	Package_S0:SOP-4_4.4x2.6mm_P1.27mm	bei DIP-Variante (AQY216 DIP): Package_DIP:DIP-4_W7.62mm
F1	Fuse:Fuse_5x20mm_Horizontal_ReferenceFuseHolder	oder gewählter Halter
RV1	Varistor:RV_Disc_D12mm_W3.9mm_P7.5mm	S14K275 passt
R1, C1, C2	Standard-THT	ok
J1-J3	Pinheader/Klemme	J2 alternativ als 4 große Lötpads
J4	Connector_PinSocket_2.54mm:PinSocket_1x04_P2.54mm_Vertical	Buchsenleiste!
U1	— (leer)	selbst anlegen , siehe 7.3

7.3 Footprint für den ESP32-C3 Super Mini anlegen

1. Footprint-Editor öffnen → neue Bibliothek `timer_project.pretty` (im Projektordner, Tabelle „projektspezifisch“).
2. Neuer Footprint `ESP32-C3_SuperMini`: zwei Padreihen 1×8, RM 2,54, Reihenabstand 15,24 mm (= 6 × 2,54), Pads ø1,7/Bohrung 1,0. Umriss 18 × 24 mm auf F.Fab, USB-Seite markieren.

3. Padnummern gemäß Pinout-Bild vergeben (linke Reihe 5V, G, 3.3, 4, 3, 2, 1, 0 / rechte Reihe 5, 6, 7, 8, 9, 10, 20, 21) und im Schaltplan dem Symbol U1 zuweisen (die Symbol-Pinnamen 5V/G/3V3/3/4/5/6/8/9 müssen auf die richtigen Padnummern zeigen).

7.4 Platine aus dem Schaltplan erzeugen

Schaltplan-Editor → **Werkzeuge** → **Schaltplan mit Platine abgleichen** („Update PCB from Schematic“). Alle Footprints landen als Haufen neben der (noch leeren) Platine.

7.5 Kontur und Referenzen importieren

1. PCB-Editor → **Datei** → **Importieren** → **Grafik** → hardware/platine_original_geometrie.dxf.
2. Einheit steht dank DXF-Header automatisch auf **mm**; Import zunächst auf Layer **User.Drawings**, Position (0,0), Maßstab 1.
3. **Maßkontrolle**: Mit dem Messwerkzeug T1→T2 messen — muss **25,6 mm** ergeben. Stimmt es, weiter; sonst Import-Einstellungen prüfen.
4. Die Außenkontur markieren (**eine geschlossene Linie** — die 6 randoffenen Dom-Schlitzte sind bereits Teil der Kontur, keine separaten Kreise mehr) → Eigenschaften → Layer auf **Edge.Cuts** ändern. Alles andere (Taster-Kreuze, Stellflächen, Fenster, Keepouts, Texte) bleibt auf User-Layern; die Stellflächen-Rahmen samt Beschriftung optional zusätzlich auf **F.Silkscreen** kopieren (hilft beim Bestücken).
5. Die 4 Befestigungslöcher als Footprints setzen: MountingHole:MountingHole_3.2mm exakt auf die grünen Kreuze (Position per **E** → Koordinaten aus Kap. 4.3 eintippen).

7.6 Platzieren und Routen

1. **Taster zuerst**: SW1-SW3 exakt auf (13,2/12,3), (38,8/12,2), (64,1/12,0) — Position numerisch eingeben, nicht schieben.
2. J4 auf die Pinkreuze (erster Pin = GND auf x 15,0 / y 27,84).
3. PS1/TSP aufs Stellflächen-Rechteck (AC-Pins Richtung linker Rand), U1/ESP auf seine Fläche (USB zur Oberkante), J2-Pads an den Rand der Shelly-Stellfläche, J1 + F1 + RV1 in die 230-V-Ecke unten links, U2/PhotoMOS zwischen ESP-Bereich und Shelly-Pads, R1 daneben, C1/C2 an die 5-V-Leitung nahe U1.
4. **Netzklassen** anlegen (Datei → Platineneinstellungen → Netzklassen): - HV (Netze L_IN, L_F, N, SW_SHELLY, O_LAST): Leiterbahn $\geq 0,8$ mm (Versorgungspfad des Shelly-Ausgangs O ≥ 2 mm bei 8-A-Last!), Abstand innerhalb HV $\geq 1,0$ mm. - Standard (Kleinspannung): 0,25/0,2 mm.
5. **Die goldene Regel**: Zwischen jedem HV-Netz und jedem Kleinspannungs-Netz $\geq 6,0$ mm **Abstand** — am einfachsten per benutzerdefinierter Regel (Platineneinstellungen → Benutzerdefinierte Regeln): (version 1) (rule HV_zu_LV (condition "A.NetClass == 'HV' && B.NetClass != 'HV'") (constraint clearance (min 6.0mm))) Einzige erlaubte „Annäherung“ ist der PhotoMOS selbst — dessen Gehäuse ist die Trennstelle (Pins 1/2 = LV, Pins 3/4 = HV).
6. Keine Masseflächen/Zonen im HV-Bereich; unter dem TSP-Modul keine Fremdleiterbahnen.
7. **DRC** (Design Rules Check) fehlerfrei bekommen.

7.7 Fertigungsdaten und Bestellung

1. **Datei** → **Fertigungsdaten** → **Gerber** (Standardlayer) + Bohrdateien.
2. 3D-Ansicht (Alt+3) als letzte Sichtprüfung.
3. Hersteller (JLCPCB/PCBWay/Aisler): 2 Lagen, 1,6 mm FR4, HASL bleifrei oder ENIG, Farbe egal. Die Außenkontur inkl. der 6 randoffenen Dom-Schlitzte kommt automatisch aus Edge.Cuts.
4. **Vor dem Absenden**: 1:1-Papierdruck der Platine (Datei → Plotten → PDF, Maßstab 1:1) ausschneiden und im Gehäuse anprobieren!

8. Montage und Inbetriebnahme

1. **Kleinteile bestücken** (Reihenfolge flach → hoch): R1, C2, U2, Taster (exakt!), J4-Buchsenleiste, C1, F1-Halter, RV1, Klemmen.
 2. **ESP32-C3 vorbereiten**: per USB flashen (Kap. 6), WLAN-Verbindung testen, dann auf die Platine löten (Stiftleisten oder direkt).
 3. **Nur-USB-Test**: ESP über USB versorgen (kein Netz!) — Taster müssen Countdown starten, OLED auf J4 aufgesteckt muss anzeigen, GPIO6 muss schalten (Messgerät/LED am PhotoMOS-Eingang).
 4. **TSP-05 einlöten**, Shelly mit VHB auf die Stellfläche kleben, vier Litzen zu J2 (L, N, SW, O), Litze zum Lastabgang.
 5. **Sichtprüfung + Durchgangsprüfung**: kein Schluss zwischen L/N/PE, ≥ 6 mm Abstände eingehalten, keine Lötspritzer.
 6. **Erster Netztest**: Gehäuse geschlossen, über PRCD/RCD einschalten. OLED zeigt Uhr/Status → Shelly per Knopf/App einrichten (WLAN, Input Mode „Switch“) → Tastertest mit Last.
 7. Zeiten nach Wunsch über `http://feeder-relais.local` einstellen.
-

9. Dateiübersicht

Datei	Inhalt	Regenerierbar?
hardware/platine_template.py	Quelle der Wahrheit für alle Geometrie; parametrisch	Quelle
hardware/ platine_original_geometrie.dxf	daraus erzeugte DXF für KiCad	ja: python3 platine_template.py (benötigt pip install ezdxf)
hardware/platine_vorschau.png	Render der DXF	ja (Renderskript siehe CLAUDE.md)
hardware/ timer_ersatzplatine.kicad_sch	kompletter Schaltplan (KiCad 8+)	Hand-gepflegt
firmware/timer-relais-c3.yaml	ESPHome-Konfiguration	Hand-gepflegt
firmware/timer_web.h	Mobile Web-App + JSON-API (C++-Handler auf web_server_base)	Hand-gepflegt
firmware/net_config.h	Persistente Netzwerk-Konfig (IP-Modus, statische IP, NTP, Hostname, 802.11k/v-Roaming)	Hand-gepflegt
firmware/log_ring.h	Log-/Debug-Ringpuffer für den Service-Tab (/api/log)	Hand-gepflegt
firmware/build/*.bin	fertige Flash-Images (factory + ota), aktueller Stand	generiert
firmware/build_images.sh	kompiliert und aktualisiert firmware/build/	Quelle
docs/DOKUMENTATION.md	dieses Dokument (deutsche Quelle , maßgeblich)	Hand-gepflegt
docs/DOKUMENTATION.en.md, docs/ DOKUMENTATION.fr.md	Handbuch in Englisch/ Französisch (Übersetzung, je Release nachgezogen)	Hand-gepflegt

Datei	Inhalt	Regenerierbar?
docs/PROJEKTPLAN.md	Phasen, Status, Checklisten (nur Deutsch)	Hand-gepflegt
CLAUDE.md	Arbeitsanweisung für Claude Code	Hand-gepflegt
docs/DOKUMENTATION.pdf, docs/PROJEKTPLAN.pdf	PDF-Fassungen (Pflicht bei jeder Änderung)	ja: python3 tools/md2pdf.py docs/*.md
tools/md2pdf.py	Markdown→PDF-Generator (bettet Bilder ein)	Quelle
docs/img/*.png	Diagramme der Flash-Anleitung (Kap. 6.1)	generiert (PIL)

10. Änderungshistorie der Geometrie-Vorlage

Version	Änderung
v1	Erste Koordinaten aus perspektivisch entzerrten Fotos ($\pm 1-2$ mm)
v2	Scanner-Vermessung: $76,7 \times 66,1$, Taster/Löcher/Aussparung präzise (Lötseiten-Scan)
v3	Gehäuse-Einpassung $105,3 \times 79,4$, 6 Dom-Ausschnitte $\varnothing 10$, alte Aussparung entfällt (= B5)
v4	Korrektur: kein Versenken möglich (unten nur 1,8 mm) — Fenster raus, Shelly auf Stellfläche rechts, 4. Loch zurück, Steg-Keepouts
v5	TSP-05 statt HLK eingeplant (unten links), ESP nach oben links
v6	OLED rahmenmontiert: Fensterbereich freigegeben, J4-Anschluss
v7	Beschriftete Platzhalter (DXF-Texte)
v8	OLED-Steckmontage: J4 senkrecht auf Fensterzentrierung konstruiert
v9	Dompositionen direkt vermessen (Silvio 13.07.: Pitch 45, Reihenabstand 70, obere Linie 6 mm über Ober-, linke Spalte 6 mm neben Linkskante $\rightarrow x -6/39/84$, $y -6/+64$); 6 Dom-Ausschnitte von $\varnothing 10$ -Bohrungen auf randoffene U-Schlitze umgestellt (toleranter beim Einsetzen); Eckradien an den äußeren Schlitzen lokal auf 3,2/2,1 reduziert; Kontur bleibt geschlossene Edge.Cuts-Schleife (verifiziert)